Beautiful Soup — Complete Python Notes

1. What Is Beautiful Soup?

Beautiful Soup is a Python library used to parse HTML and XML documents and extract data from them.

It helps us search, navigate, and modify HTML documents.

Uses of Beautiful Soup

- Extract links from webpages.
- Extract text, headings, and paragraphs.
- Extract image URLs.
- Find HTML elements by tag, class, or ID.
- Extract product information from HTML.
- Navigate parent, child, and sibling elements.
- Parse HTML tables.
- Modify HTML documents.
- Clean and transform HTML.

Important: Beautiful Soup does not download webpages by itself. We commonly use `requests` to download webpage content and Beautiful Soup to parse it.

2. Installing Beautiful Soup

Install Beautiful Soup:

```bash
```

pip install beautifulsoup4
```

Install Requests for downloading webpages:

```bash
pip install requests beautifulsoup4
```

Import the libraries:

```python
import requests
from bs4 import BeautifulSoup
```

Note: The package name is `beautifulsoup4`, but the import name is `bs4`.

3. Basic Web Scraping Workflow

A typical web scraping workflow consists of four steps:

1. Send an HTTP request.
2. Receive HTML content.
3. Parse the HTML using Beautiful Soup.
4. Extract the required data.

Example
```

```python
import requests
from bs4 import BeautifulSoup

url = "https://example.com"

response = requests.get(url, timeout=15)
response.raise_for_status()

soup = BeautifulSoup(response.text, "html.parser")

print(soup.title)
```

Explanation

- `requests.get()` sends an HTTP GET request.
- `timeout=15` sets a request timeout.
- `response.raise_for_status()` raises an exception for HTTP 4xx and 5xx responses.
- `response.text` retrieves the response body as text.
- `BeautifulSoup()` parses the HTML.
- `soup.title` retrieves the title element.

4. Understanding HTML Structure

Example HTML

```html
<!DOCTYPE html>
```

```
<html>
<head>
    <title>My Website</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph.</p>
    <a href="https://example.com">Visit Website</a>
    <img src="image.png" alt="Example Image">
</body>
</html>
```

Important HTML Elements

Element or Attribute	Description
`<html>`	Root HTML element
`<head>`	Contains document metadata
`<title>`	Defines the document title
`<body>`	Contains the main page content
`<h1>`	Main heading
`<p>`	Paragraph
`<a>`	Hyperlink
`<img>`	Image
`href`	Link destination
`src`	Resource URL
`class`	CSS class
`id`	Unique element identifier

5. Creating a BeautifulSoup Object

Beautiful Soup can parse an HTML string directly.

```python
from bs4 import BeautifulSoup

html = """
<html>
<body>
    <h1>Python Notes</h1>
    <p>Learning Python</p>
    <p>Python is easy to learn.</p>
</body>
</html>
"""

soup = BeautifulSoup(html, "html.parser")

print(soup.prettify())
```

What Is `prettify()`?

`prettify()` returns a formatted representation of the parsed HTML with indentation and line breaks.

```python
```

print(soup.prettify())
```

It is useful for inspecting and debugging HTML.

## 6. Beautiful Soup Object Types

Beautiful Soup commonly works with four types of objects.

| Object | Description |
|---|---|
| `BeautifulSoup` | Represents the parsed document |
| `Tag` | Represents an HTML or XML element |
| `NavigableString` | Represents a text node |
| `Comment` | Represents an HTML comment |

### Example

```python
from bs4 import BeautifulSoup

html = """
<html>
<body>
    <h1>Hello Python</h1>
</body>
</html>
"""

```
soup = BeautifulSoup(html, "html.parser")

print(type(soup))
print(type(soup.h1))
print(type(soup.h1.string))
```

7. Accessing HTML Elements Directly

You can access the first matching element by its tag name.

```python
print(soup.title)
print(soup.head)
print(soup.body)
print(soup.h1)
```

Example

```python
html = "<h1>Hello Python</h1>"

soup = BeautifulSoup(html, "html.parser")

print(soup.h1)
```

Output:

```html
<h1>Hello Python</h1>
```

This method is convenient when you need the first matching element.

8. Using `find()`

The `find()` method returns the first matching element or `None` if no match exists.

Syntax

```python
soup.find(name, attrs, recursive, string, **kwargs)
```

Example

```python
heading = soup.find("h1")

print(heading)
```

Extract Text

```python
if heading:
```

```
    print(heading.get_text(strip=True))
```

Output:

```text
Hello Python
```

Finding by ID

```python
element = soup.find(id="main-content")
```

Finding by Class

```python
element = soup.find("div", class_="product")
```

9. Using `find_all()`

The `find_all()` method finds all elements matching the specified criteria.

Syntax

```python
soup.find_all(name, attrs, recursive, string, limit, **kwargs)
```

```

### Example

```python
html = """
<h2>Python</h2>
<h2>Django</h2>
<h2>Scrapy</h2>
"""

soup = BeautifulSoup(html, "html.parser")

headings = soup.find_all("h2")

for heading in headings:
    print(heading.get_text(strip=True))
```

Output:

```text
Python
Django
Scrapy
```

Limit the Number of Results
```

```python
links = soup.find_all("a", href=True, limit=5)
```

This returns at most five matching elements.

10. Difference Between `find()` and `find_all()`

Feature	`find()`	`find_all()`
Result	First matching element	All matching elements
Return type	`Tag` or `None`	`ResultSet`
No match	Returns `None`	Returns an empty result set
Typical use	Find one title	Extract all links

Example

```python
first_link = soup.find("a")

all_links = soup.find_all("a")
```

11. Extracting Text

The `get_text()` method extracts text from an element and its descendants.

Example

```python
html = """
<div>
    <h1>Beautiful Soup</h1>
    <p>Python web scraping library.</p>
</div>
"""

soup = BeautifulSoup(html, "html.parser")

print(soup.get_text())
```

Output:

```text
Beautiful Soup
Python web scraping library.
```

Using `strip=True`

```python
text = soup.p.get_text(strip=True)

print(text)
```

This removes whitespace around the extracted text pieces.

Using `separator`

```python
text = soup.div.get_text(separator=" ", strip=True)

print(text)
```

The separator controls how text pieces are joined.

Using `.string`

```python
html = "<h1>Hello Python</h1>"

soup = BeautifulSoup(html, "html.parser")

print(soup.h1.string)
```

Output:

```text
Hello Python
```

`.string` is useful when an element has a single string child. It may return `None` for elements with multiple or complex children.

12. Extracting HTML Attributes

HTML attributes provide additional information about elements.

Example HTML

```html
<a href="https://python.org" class="website">Python</a>
```

Access an Attribute

```python
link = soup.find("a")

print(link["href"])
print(link["class"])
```

Using `.get()`

```python
print(link.get("href"))
```

Difference

- `link["href"]` raises `KeyError` if the attribute is missing.

- `link.get("href")` returns `None` by default if the attribute is missing.

Extract Image Attributes

```python
html = """
<img src="python.png" alt="Python Logo">
"""

soup = BeautifulSoup(html, "html.parser")

image = soup.find("img")

print(image.get("src"))
print(image.get("alt"))
```

Output:

```text
python.png
Python Logo
```

Check Whether an Attribute Exists

```python
if image.has_attr("src"):
    print(image["src"])
```

```

### Access All Attributes

```python
print(image.attrs)
```

The `attrs` property provides a dictionary-like mapping of an element's attributes.

13. Searching by Class

HTML

```html
<div class="product">Laptop</div>
<div class="product">Keyboard</div>
<div class="footer">Contact Us</div>
```

Python

```python
products = soup.find_all("div", class_="product")

for product in products:
    print(product.get_text(strip=True))
```
```

Output:

```text
Laptop
Keyboard
```

The argument is `class_` because `class` is a reserved keyword in Python.

Searching for Multiple Classes

```python
elements = soup.find_all("p", class_="article")
```

For more complex combinations, CSS selectors are often easier.

14. Searching by ID

HTML

```html
<div id="main-content">Welcome to Python</div>
```

Python

```python
element = soup.find("div", id="main-content")
```

````
if element:
    print(element.get_text(strip=True))
```

You can also use a CSS selector:

```python
element = soup.select_one("#main-content")
```

In CSS selectors, `#` identifies an ID.

## 15. CSS Selectors

Beautiful Soup supports CSS selectors through `select()` and `select_one()`.

| Selector | Meaning |
|---|---|
| `"p"` | All paragraph elements |
| `"a"` | All anchor elements |
| `".product"` | Elements with class `product` |
| `"#main"` | Element with ID `main` |
| `"div.product"` | `div` elements with class `product` |
| `"div p"` | Paragraphs inside a `div` |
| `"div > p"` | Paragraphs that are direct children of a `div` |
| `"a[href]"` | Anchor elements with an `href` attribute |
| `"input[type='text']"` | Text input elements |
| `"ul li"` | List items inside unordered lists |

### Using `select()`

```python
products = soup.select(".product")

for product in products:
    print(product.get_text(strip=True))
```

### Using `select_one()`

```python
product = soup.select_one(".product")

if product:
    print(product.get_text(strip=True))
```

`select()` returns all matches, while `select_one()` returns the first matching element or `None`.

## 16. Navigating the HTML Tree

HTML elements form a nested tree. Beautiful Soup provides properties and methods to navigate this structure.

### Example HTML

```html
```

```
<div id="parent">
    <h2>Title</h2>
    <p>First paragraph</p>
    <p>Second paragraph</p>
</div>
```

### Parent

```python
paragraph = soup.find("p")

print(paragraph.parent.name)
```

### Children

```python
parent = soup.find(id="parent")

for child in parent.children:
    print(child)
```

`.children` iterates over immediate child nodes, including whitespace strings.

### Descendants

```python
```

```
for element in parent.descendants:
    print(element)
```

`.descendants` traverses all nested elements and text nodes.

### Next Matching Sibling

```python
first_paragraph = soup.find("p")

next_paragraph = first_paragraph.find_next_sibling("p")

print(next_paragraph)
```

### Previous Matching Sibling

```python
paragraphs = soup.find_all("p")

if len(paragraphs) >= 2:
    previous = paragraphs[1].find_previous_sibling("p")
    print(previous)
```

### Other Navigation Properties

- `.parent` — Immediate parent.

- `.parents` — Iterates through ancestors.
- `.children` — Immediate children.
- `.descendants` — All descendants.
- `.next_sibling` — Next sibling node.
- `.previous_sibling` — Previous sibling node.
- `.next_element` — Next node in document traversal order.
- `.previous_element` — Previous node in document traversal order.

Sibling properties may return whitespace strings. Use methods such as `find_next_sibling("p")` when you need a specific tag.

## 17. Extracting All Links

### Example

```python
for link in soup.find_all("a", href=True):
    print(link["href"])
```

The `href=True` argument selects anchor tags that have an `href` attribute.

### Extract Link Text and URL

```python
for link in soup.find_all("a", href=True):
    text = link.get_text(strip=True)
    url = link["href"]
```

```
    print("Text:", text)
    print("URL:", url)
```

### Converting Relative URLs to Absolute URLs

```python
from urllib.parse import urljoin

base_url = "https://example.com"

for link in soup.find_all("a", href=True):
    absolute_url = urljoin(base_url, link["href"])
    print(absolute_url)
```

Use the appropriate page URL as the base, especially if redirects occurred.

## 18. Extracting All Images

### Extract Image Sources

```python
for image in soup.find_all("img", src=True):
    print(image["src"])
```

### Extract Sources and Alternative Text

```python
for image in soup.find_all("img"):
    print("Source:", image.get("src"))
    print("Alt:", image.get("alt"))
```

### Convert Image URLs to Absolute URLs

```python
from urllib.parse import urljoin

base_url = "https://example.com"

for image in soup.find_all("img", src=True):
    image_url = urljoin(base_url, image["src"])
    print(image_url)
```

Some websites use lazy loading, `srcset`, or JavaScript to load images. The actual URL may be stored in another attribute or may not appear in the original HTML.

## 19. Extracting HTML Tables

### Example HTML

```html
<table>
    <tr>
        <th>Name</th>
```

```
        <th>Type</th>
    </tr>
    <tr>
        <td>Python</td>
        <td>Language</td>
    </tr>
    <tr>
        <td>Django</td>
        <td>Framework</td>
    </tr>
</table>
```

### Python

```python
table = soup.find("table")

if table:
    for row in table.find_all("tr"):
        cells = row.find_all(["th", "td"])

        values = [
            cell.get_text(strip=True)
            for cell in cells
        ]

        print(values)
```

Output:

```python
['Name', 'Type']
['Python', 'Language']
['Django', 'Framework']
```

You can store the extracted table data in lists, dictionaries, CSV files, or a database.

## 20. Extracting Data from Product Cards

### Example HTML

```html
<div class="product">
    <h2 class="name">Laptop</h2>
    <span class="price">$800</span>
</div>

<div class="product">
    <h2 class="name">Keyboard</h2>
    <span class="price">$50</span>
</div>
```

### Python

```python
products = soup.find_all("div", class_="product")

for product in products:
    name_tag = product.find("h2", class_="name")
    price_tag = product.find("span", class_="price")

    name = (
        name_tag.get_text(strip=True)
        if name_tag
        else "Unknown"
    )

    price = (
        price_tag.get_text(strip=True)
        if price_tag
        else "Not available"
    )

    print(name, price)
```

Output:

```text
Laptop $800
Keyboard $50
```

Always check whether the expected elements exist before accessing their text.

## 21. Difference Between `.get_text()`, `.string`, and `.text`

| Method or Property | Purpose |
|---|---|
| `.get_text()` | Extracts text from an element and its descendants |
| `.get_text(strip=True)` | Extracts text and strips whitespace around text pieces |
| `.string` | Returns a single string child when available |
| `.text` | Convenient alias for extracting descendant text |

### Example

```python
html = "<p>Hello <b>Python</b></p>"

soup = BeautifulSoup(html, "html.parser")

print(soup.p.get_text())
print(soup.p.text)
print(soup.p.string)
```

Output:

```text
Hello Python
Hello Python
None
```

```

In this example, `.string` returns `None` because the paragraph contains multiple children.

## 22. Modifying HTML

Beautiful Soup can modify the parsed document in memory.

### Change Text

```python
heading = soup.find("h1")

if heading:
    heading.string = "New Heading"
```

### Change an Attribute

```python
link = soup.find("a")

if link:
    link["href"] = "https://python.org"
```

### Add a New Tag
```

```python
new_tag = soup.new_tag("p", id="new-paragraph")

new_tag.string = "This is a new paragraph."

if soup.body:
    soup.body.append(new_tag)
```

### Remove a Tag and Its Contents

```python
tag = soup.find("script")

if tag:
    tag.decompose()
```

### Remove a Wrapper but Keep Its Contents

```python
tag = soup.find("span")

if tag:
    tag.unwrap()
```

### Replace a Tag

```python
old_tag = soup.find("b")

if old_tag:
    new_tag = soup.new_tag("strong")
    new_tag.string = old_tag.get_text()

    old_tag.replace_with(new_tag)
```

### Display the Modified HTML

```python
print(soup.prettify())
```

These changes affect the parsed document in memory. They do not automatically update the original website.

## 23. Parsing Different Types of Content

Beautiful Soup supports HTML and XML parsing.

### Using Python's Built-in HTML Parser

```python
soup = BeautifulSoup(html, "html.parser")
```

### Using `lxml`

Install:

```bash
pip install lxml
```

Use:

```python
soup = BeautifulSoup(html, "lxml")
```

### Using `html5lib`

Install:

```bash
pip install html5lib
```

Use:

```python
soup = BeautifulSoup(html, "html5lib")
```

### Parser Comparison

| Parser | Description |

|---|---|

| `html.parser` | Built into Python |

| `lxml` | Requires the `lxml` package |

| `html5lib` | Aims to parse HTML according to browser-like HTML5 rules |

| `xml` | XML parsing mode; requires appropriate parser support |

The best parser depends on the document format, malformed HTML, and your project requirements.

## 24. Handling Missing Elements Safely

### Incorrect

```python
title = soup.find("h1").get_text(strip=True)
```

If no matching element exists, `find()` returns `None`, resulting in an `AttributeError`.

### Correct

```python
heading = soup.find("h1")

if heading:
    title = heading.get_text(strip=True)
else:
```

```
    title = "No heading found"

print(title)
```

### For Product Cards

```python
for product in soup.select(".product"):
    name_tag = product.select_one(".name")
    price_tag = product.select_one(".price")

    name = (
        name_tag.get_text(strip=True)
        if name_tag
        else "Unknown"
    )

    price = (
        price_tag.get_text(strip=True)
        if price_tag
        else "Not available"
    )

    print(name, price)
```

This makes your scraper more robust when HTML fields are missing.

## 25. Complete Web Scraping Project

This program extracts a page title, headings, paragraphs, and links.

```python
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin

url = "https://example.com"

try:
    response = requests.get(url, timeout=15)
    response.raise_for_status()

    soup = BeautifulSoup(response.text, "html.parser")

    # Extract page title
    title_tag = soup.find("title")

    title = (
        title_tag.get_text(strip=True)
        if title_tag
        else "No title found"
    )

    print("Page Title:", title)
    print("-" * 40)
```

```
    # Extract headings
    for heading in soup.find_all(["h1", "h2", "h3"]):
        print(
            heading.name,
            heading.get_text(strip=True)
        )

    print("-" * 40)

    # Extract paragraphs
    for paragraph in soup.find_all("p"):
        print(paragraph.get_text(strip=True))

    print("-" * 40)

    # Extract links
    for link in soup.find_all("a", href=True):
        text = link.get_text(strip=True)
        href = urljoin(response.url, link["href"])

        print(text, "->", href)

except requests.exceptions.RequestException as error:
    print("Request error:", error)
```

This example demonstrates a reusable workflow for web scraping. The output depends on the actual HTML returned by the website.

## 26. Common Errors and Solutions

| Error or Problem | Solution |
|---|---|
| `ModuleNotFoundError: No module named 'bs4'` | Install `beautifulsoup4` in the active Python environment |
| `ModuleNotFoundError: No module named 'requests'` | Install `requests` |
| `AttributeError: 'NoneType' object has no attribute ...` | Check whether the element exists before accessing it |
| `KeyError: 'href'` | Check the attribute or use `.get()` |
| `HTTPError` | Inspect the HTTP status and handle the response |
| `Timeout` | Set a suitable timeout and retry carefully if appropriate |
| No elements found | Inspect the received HTML and verify the selectors |
| Dynamic content missing | Determine whether JavaScript generates the content |
| Incorrect relative URLs | Use `urljoin()` with the correct base URL |

## 27. Beautiful Soup vs Requests vs Selenium vs Scrapy

| Tool | Main Purpose |
|---|---|
| Requests | Fetch web resources over HTTP |
| Beautiful Soup | Parse HTML/XML and extract data |
| Selenium | Automate a browser and interact with rendered pages |
| Playwright | Automate browsers and JavaScript-driven pages |
| Scrapy | Build larger crawling and scraping projects |

A common combination is Requests + Beautiful Soup for static HTML.

For pages where the required content is generated by JavaScript, browser automation may be more appropriate.

## 28. Best Practices

1. Use request timeouts.
2. Call `raise_for_status()` before parsing the response.
3. Handle missing elements and attributes safely.
4. Use CSS selectors when they simplify complex queries.
5. Convert relative URLs using `urljoin()`.
6. Avoid unnecessary repeated requests.
7. Respect website terms, access restrictions, and applicable crawling rules.
8. Avoid excessive request rates.
9. Use browser automation only when necessary.
10. Separate downloading, parsing, extraction, and saving logic.

## 29. Interview Questions and Answers

### Q1. What is Beautiful Soup?

Beautiful Soup is a Python library for parsing HTML or XML documents and extracting information.

### Q2. Does Beautiful Soup download webpages?

No. It parses content supplied to it. Requests is commonly used to download webpages.

### Q3. What is a BeautifulSoup object?

It represents the parsed document and provides methods for searching and navigating its structure.

### Q4. What is the difference between `find()` and `find_all()`?

`find()` returns the first matching element or `None`. `find_all()` returns all matching elements in a `ResultSet`.

### Q5. What is the purpose of `select()`?

It finds all elements matching a CSS selector.

### Q6. What does `get_text(strip=True)` do?

It extracts text and strips whitespace around individual text pieces.

### Q7. How do you extract all links?

```python
for link in soup.find_all("a", href=True):
    print(link["href"])
```

### Q8. How do you find an element by ID?

```python
soup.find(id="main")
```

Alternatively:

```python
```

```
soup.select_one("#main")
```

### Q9. How do you find elements by class?

```python
soup.find_all(class_="product")
```

Alternatively:

```python
soup.select(".product")
```

### Q10. Can Beautiful Soup execute JavaScript?

No. Beautiful Soup parses the supplied markup but does not execute JavaScript.

### Q11. What is `prettify()`?

It returns formatted markup with indentation and line breaks.

### Q12. What is the difference between `get()` and bracket-based attribute access?

`tag.get("href")` returns `None` by default when the attribute is missing, whereas `tag["href"]` raises `KeyError`.

## 30. Practice Exercises

1. Print the title of an HTML page.
2. Extract all paragraph elements.
3. Extract every link URL.
4. Find all elements with the class `product`.
5. Find the element with ID `content`.
6. Extract all image sources.
7. Extract HTML table rows as lists of text.
8. Convert relative URLs into absolute URLs.
9. Extract product names and prices.
10. Modify an HTML heading.
11. Remove all script elements from a parsed document.
12. Save extracted links to a text file.

### Example Solutions

**1. Extract the page title**

```python
title = soup.find("title")

if title:
    print(title.get_text(strip=True))
```

**2. Extract all paragraphs**

```python
for paragraph in soup.find_all("p"):
```

```
    print(paragraph.get_text(strip=True))
```

**3. Extract every link URL**

```python
for link in soup.find_all("a", href=True):
    print(link["href"])
```

**4. Find all products**

```python
for product in soup.select(".product"):
    print(product.get_text(strip=True))
```

**5. Find the element with ID `content`**

```python
element = soup.find(id="content")

if element:
    print(element.get_text(strip=True))
```

**6. Extract image sources**

```python
```

```
for image in soup.find_all("img", src=True):
    print(image["src"])
```

**7. Extract table rows**

```python
table = soup.find("table")

if table:
    for row in table.find_all("tr"):
        values = [
            cell.get_text(strip=True)
            for cell in row.find_all(["th", "td"])
        ]

        print(values)
```

**8. Convert relative URLs into absolute URLs**

```python
from urllib.parse import urljoin

base_url = "https://example.com"

for link in soup.find_all("a", href=True):
    print(urljoin(base_url, link["href"]))
```

**9. Extract product names and prices**

```python
for product in soup.select(".product"):
    name = product.select_one(".name")
    price = product.select_one(".price")

    print(
        name.get_text(strip=True) if name else "Unknown",
        price.get_text(strip=True) if price else "Not available"
    )
```

**10. Modify an HTML heading**

```python
heading = soup.find("h1")

if heading:
    heading.string = "New Heading"

print(soup.prettify())
```

**11. Remove all script elements**

```python
for script in soup.find_all("script"):
```

```
    script.decompose()
```

**12. Save extracted links to a text file**

```python
with open("links.txt", "w", encoding="utf-8") as file:
    for link in soup.find_all("a", href=True):
        file.write(link["href"] + "\n")
```

## 31. Beautiful Soup Cheat Sheet

| Task | Code |
|---|---|
| Create parser | `BeautifulSoup(html, "html.parser")` |
| Find first tag | `soup.find("p")` |
| Find all tags | `soup.find_all("p")` |
| Find by class | `soup.find_all(class_="item")` |
| Find by ID | `soup.find(id="main")` |
| CSS selector | `soup.select(".item")` |
| First CSS match | `soup.select_one("#main")` |
| Extract text | `tag.get_text(strip=True)` |
| Extract attribute | `tag.get("href")` |
| Check attribute | `tag.has_attr("href")` |
| Format HTML | `soup.prettify()` |
| Find parent | `tag.parent` |
| Iterate children | `tag.children` |
| Find descendants | `tag.descendants` |

| Resolve relative URL | `urljoin(base_url, relative_url)` |

| Remove element and contents | `tag.decompose()` |

| Remove wrapper, keep contents | `tag.unwrap()` |

## Final Summary

Beautiful Soup parses HTML into a searchable tree. You can use its methods and CSS selectors to locate elements, extract text and attributes, navigate relationships, and modify the parsed document.

The core workflow is:

Download HTML → Parse HTML → Find Elements → Extract Data → Process or Save Results.

For a Python backend developer, learning Requests and Beautiful Soup provides a strong foundation for understanding web scraping before moving on to Scrapy and browser automation tools.